

摘要：

YC掌门人借助Claude Code重返编程，他调度15个AI代理并行作业，在全职CEO工作之余产出数十万行代码。他的核心秘诀：别省token，像交房租一样烧；别用代码写该用语言描述的逻辑；先画ASCII图再动手。400倍效率差距背后，是一套任何人都能复制的工作流。他同时警示：个人AI时代即将到来，关键问题是——你掌控工具，还是被工具掌控？

一个13年没写过代码的投资人，重新坐到键盘前，5天、200美元，重建了曾经花掉400万美元和一年半时间的产品——这件事到底是怎么发生的？

近日，Y Combinator在播客节目中发布了一期特别访谈。YC总裁Gary Tan与主持人详细复盘了他过去数月的“复出”经历：在中断编程整整13年后，他借助AI编程工具Claude Code重新开始写代码，并在保持YC全职CEO工作的同时，产出了数十万行代码，多个开源项目在GitHub上从零积累到超过10万颗星。

Gary Tan在节目中说：“我自己也很震惊。13年没写代码，然后突然——砰，我的产出是当年的400倍。”

同时他警示，个人AI时代即将到来，但一个关键问题是——你掌控工具，还是被工具掌控？



## 5天重建一个“死去”的创业项目

故事要从Gary的第一个YC创业项目说起。

2008年，他做了一个叫Posterous的极简博客平台——“用邮件写博客”。这个产品最终长到了全球前200大网站，被Twitter以约2000万美元收购。后来Twitter关掉了它。Gary没钱从Twitter买回来（要价几百万美元）。

近日，他打开了Claude Code，开始重建这个他2008年创办的第一个YC项目——Posterous。这个平台他已经建过三次了：

- 。第一次（2008年）：花了约400万美元，六七个人，历时一年半
- 。第二次（Post Haven）：约10万美元，两个人，约三个月

- **第三次（2025年1月）**：约200美元（Claude Code Max账户费用），**五天**，功能完整，还附带了RAG检索、深度研究、自动爬取等AI能力

**200美元对比400万美元。五天对比一年半。**

## “400倍”从哪里来？

Gary Tan最初在社交媒体上说自己的**编码速度是2013年的“100倍”**，随即遭到大量质疑——“**代码行数不等于生产力**”。

他随后做了一件更较真的事：用公开的Git工具对代码进行标准化处理，剔除注释、空行等冗余内容，**只统计“逻辑代码行数”**。结果出乎他自己意料——数字不降反升，从100倍变成了**400倍**。

这个数字怎么理解？Gary Tan给出了参照系：根据软件工程领域的文献，一名专业工程师每天能产出的经过测试、可投入生产的代码，大约是**30到50行**。Gary Tan 2013年兼职写代码时，大约是每天14行。

**而现在，他同时调度15个AI代理并行工作，在过去48小时内合并了13个Pull Request。**

他说，“我不是在写代码，我是在指挥15个代理去写……但那些代码是真实的、经过测试的、可以运行的。”

## 工作流实录：他是怎么每天产出数百行代码的

节目中，Gary Tan直接展示了他的真实工作界面。他日常的工作流核心是三个工具的组合：Claude Code + Conductor + GStack。

GStack是什么？它本来不是一个“项目”，而是一个意外。Gary Tan发现自己反复在Claude Code里输入相同的指令，烦了，就把常用的指令整理成了一个markdown文件。后来这个文件越来越完善，变成了一套完整的AI编程“技能库”，最后作为开源项目发布，在GitHub上从零积累到超过10万颗星。

**他详细描述了一个具体的操作习惯：让Claude在开始写代码之前，先画一张ASCII艺术图——数据流、状态机、依赖关系图、处理流程、决策树，全部用文字符号画出来。**

“一旦它把上下文全部加载进来，它做的事情就完整得多了。”他说。

他在GStack里设置了几个核心skills：

- CEO Review：类似Brian Chesky的“10星体验”思维练习——五星太普通了，六星七星八星是什么？每次开始一个新功能，先想清楚最理想的状态是什么
- 计划模式（Plan Mode）：先规划再动手，避免AI“盲目开干”
- 自动QA：接入Microsoft Playwright，让AI在测试后自动用浏览器验证结果，实现黑盒测试
- /codex技能：遇到特别复杂的问题，从Claude Code切换到OpenAI Codex，他的比喻是：“Claude Code是适合ADHD CEO的工具，但有时候你需要那个200 IQ、几乎不开口的CTO来帮你解决真正硬的问题。”



他说：“最近48小时我提交了13个PR，想到一个新想法就直接排队，Claude去执行，我来验证。”

## “Token Maxxing”：别省，像交房租一样烧token

Gary Tan在节目中反复提到一个他自创的概念——“**Token Maxxing**”（代币最大化），意思是：不要吝啬AI的token消耗，要尽可能多地给模型喂上下文、多来源交叉验证、做更完整的研究。

他打了个比方：这就像旧金山的房租。

“我们经常跟YC创始人说，旧金山租金很贵，但不住在那里的代价更贵，”他说，“Token Maxxing也是一样。你应该尽可能多地花在模型和token上，而不是把它当成办公桌那样能省则省。”

他的Gary's List网站每篇文章的生成成本大约是5到10美元的API调用费，但系统会自动抓取数十篇文章、整本书、多个数据源，交叉比对后生成有引用来源的深度报道。“**花5到10美元，做一个人类记者要花一个月才能做完的研究，**”他说。

## “轻框架，重技能”：AI工程的底层哲学

Gary在节目中总结了他的核心工程哲学，他称之为“**Thin Harness, Fat Skills**”（轻框架，重技能）。

意思是：不要把精力花在重复搭建“框架层”（harness）上，这一层交给成熟工具就好。真正该投入的，是用自然语言写清楚“这件事应该怎么做”的markdown提示词——也就是“**skills**”。

他打了个比方：

想象你是一个婚礼策划人，你要写下一份清单，教下一个要做这件事的人怎么做。所有这些内容都应该写进markdown。而那些需要确定性执行的事，比如给20个场地打电话，你才用代码去做，比如调用Twilio的API。

他认为，很多人在AI工程上失败，正是因为把“应该用语言描述的事情”错误地用代码去实现——代码不理解特殊情况，不理解你是谁，不理解你想要什么。

## AI代理像法拉利：会跑，也会抛锚

Gary用了一个贯穿全程的比喻来形容当下的AI编程体验：

用OpenClaw现在就像开法拉利——令人窒息的兴奋，它能搞定你根本想不到机器能搞定的事，而且速度极快。但它也确实是法拉利——它会在你最需要它的时候，在路边抛锚，你得自己拿扳手，掀开引擎盖，自己修。

他把现在的时代比作“家酿计算机俱乐部”（Homebrew Computer Club）的年代——Apple 1刚出来时，就是一块面包板装进木箱子里钉钉子做的。当下的AI编程工具同样粗糙，同样充满可能性。

你需要花两三个小时，可能还要花500到1000美元的token和云服务费，才能把这东西跑起来。

但一旦跑起来，就像拥有了一辆套件法拉利——你可以去任何地方。

## 个人AI的未来：你控制工具，还是工具控制你？

Gary还抛出了一个更大的判断：

我保证，明年这个时候，地球上每一个人都会有自己的私人AI。

他认为，个人AI的普及将是继个人电脑之后最重要的技术转变。但这个转变有两种走向：一种是你拥有自己的AI，自己的数据，自己写提示词，自己掌控看到什么；另一种是企业控制的AI，就像Facebook算法一样，你不知道背后是谁写的、服务于谁的商业模式。

“这是一个定义性的问题：你会掌控自己的工具，还是你的工具会掌控你？”他说。



访谈全文如下：

### Tokenmaxxing：顶尖创业者如何用AI完成400名工程师的工作

Y Combinator | Lightcone Podcast

#### 你会掌控自己的AI吗？

Gary Tan：我觉得这是一个决定性的问题——你能掌控自己的工具，还是你的工具会掌控你？现在使用OpenClaw就像在驾驶一辆法拉利，令人肾上腺素飙升，感觉不可思议。它能搞定一些你永远不会想到机器能搞定的事情，而且速度极快。但它同时也像一辆法拉利——你最好懂点机械。它会在你最需要它的时候抛锚在路边，你得拿着扳手出来，打开引擎盖，自己动手修。这是计算机科学技术领域非常令人振奋的时代。

#### 阔别13年后重新写代码

主持人：欢迎回到Lightcone特别节目。本期我们将聊聊Gary Tan是如何重回“建造者”状态的。如果你在Twitter上关注我们，你会知道，在经历了多年的投资生涯之后，Gary回来了，重新成为一名构建者。过去几个月里，他提交了数十万行代码，构建了多个热门开源项目，这些项目从零开始，在GitHub上积累了超过10万颗Star。而这一切，都发生在他全职担任YC CEO这一极度繁忙的工作期间。网上很多人觉得这根本不可能，甚至有些难以置信——但它确实发生了。我们亲眼见证了整个过程。今天，我们就来聊聊他是怎么做到的。



Gary Tan：我自己也相当震惊。整整13年没有写代码，然后突然之间，我的代码产出量达到了当年的大约400倍。而那一年，我大概也只有三分之二的时间在写代码。

主持人：不如我们先从那个开创一切的项目讲起——Gary's List。聊聊几个月前你是怎么启动Claude Code、重新开始写代码的。是在某期Lightcone节目之后，对吧？

Gary Tan：对，就是那之后。

## 用Claude Code重建一个创业项目

Gary Tan：我意识到自己想把所有认同我理念的人聚集在一起，尤其是关于加利福尼亚州的议题。于是我成立了一个501(c)(4)组织，现在已经有有了一个C3和一个PAC，这是很多政治团体的常见运作方式。大家往往只关注资金，但我们真正想做的是把有识之士汇聚在一起。在旧金山政治圈摸爬滚打多年，我深刻体会到，凝聚人心是一股巨大的力量，这正是大众社会运动的本质。

我想，为什么不建一个网站，就从这里出发呢？先从写作开始——写我真正关心的议题。比如，我希望孩子们能在学校学到真正的知识。全球的观众可能会觉得这很奇怪，但我确实觉得荒诞：在旧金山公立学校，一个六七年级的初中生，想修代数课，曾经是不可能的，直到今天依然极其困难。

这件事对我意义深远。如果我当年在东湾公立学校读书时没能学代数，我绝无可能考入斯坦福学工程，永远不会写代码，也不会有今天的一切。所以我意识到，是时候重新写代码了。

我最终重建了Posterous——那是我2008年的第一个YC创业项目。

主持人：Posterous是什么，帮不太了解的人介绍一下？

Gary Tan：Posterous是一个极简的邮件博客平台，后来发展成为全球前200大网站之一，最终被Twitter以约2000万美元收购。那是我真正意义上的第一桶金。

Twitter收购后关闭了这个产品，我为那些我们精心招募的员工感到遗憾，于是又重建了一个版本叫Post Haven。当时从Twitter手里买回来需要几百万美元，而我那时候一分钱都没有，所以最好的办法就是自己再写一遍。

今年1月，我又把它写了第三遍。第一次花了约400万美元、六七个人、大约一年半的时间；第二次花了大概10万美元，我和联合创始人Brett Gibson（现在经营Initialized）两个人，大约三个月；这一次，花了大约200美元——也就是我的Claude Code Max账户费用，加上大概五天时间，就建成了一个功能完整的博客平台，该有的功能全都有。

不仅如此，在此之上还构建了完整的RAG系统和完整的智能体检索功能——能够出去读遍整个互联网：我所有的推文、任何话题的递归爬取与深度研究。代数问题只是我们真正关心的众多议题之一。能够摄取互联网上的信息，看到所有正反方的论点，然后在后端生成极为详尽的报告——包括所有值得引用的观点——非常有价值。

Lightcone的老听众可能记得我们早期一期关于智能体系统的节目，嘉宾是Jake Heller。Jake创立了Casetext，他描述的那套方法，和我最终为记者式长篇文章所构建的系统几乎一模一样，针对各种正在发生的时事议题。

现在任何人都可以访问garyslist.org，我们每天发布两到三篇研究充分、来源完整的文章，聚焦加州、旧金山和洛杉矶的动态，探讨如何建设更好的政府。

## 像记者一样思考的软件

主持人：我觉得大家对Gary's List有一个误解。我们一直在谈的经典模式是：你构建软件，让人们来使用它。比如你建一个博客平台，人们来写博客，也许最终开了自己的Substack，发



文章。但Gary's List不只是一个博客平台——它实际上在做一名高质量调查记者的工作。它不仅仅是记者用来发布文章的工具。

Gary Tan： 对，基本上，以花费约5到10美元Opus API调用的代价，它所完成的工作量相当于一个真实的人类——要一篇一篇地翻阅几十篇文章，通读整本相关书籍，逐一做标注。

回到Casetext的例子，Jake教给我的核心是：你要思考一个人类拿到这些信息之后会怎么做。他会去检索什么？去图书馆找什么书？在网上搜索什么？现在更棒的是，你不必止步于此——你可以调用Perplexity的API做深度研究，调用X的API做深度研究，用Grok的API在X平台上做研究效果其实非常好，然后把所有这些上下文全部抓取过来。

## "Tokenmaxxing"的兴起

Gary Tan： 这又回到了我那篇关于"煮沸海洋"的文章所阐述的理念。在构建智能体软件的当下，你不必再满足于过去人类写代码时的那种局限性——这在研究领域同样适用。如果你彻底地"煮沸海洋"，追求极致完整性——对一个人类来说可能要花一个月来做的研究——你现在只需要加大投入，多花点钱，做Tokenmaxxing，但你就应该这样做。

如果存在能让结果更完整、更出色、更贴近现实的增量性工作——比如在这类写作中，我们不满于一个来源，我们可以获取20个来源，交叉对比，发现这13个来源这么说，另外7个来源持不同意见——然后把所有这些上下文输入核心提示词，你就能做出比人类仅仅点开一个链接、看个标题就下判断更好得多的决策。

我认为，Tokenmaxxing正是你现在能做的最酷的事情。这不只是生成文章，也不只是写代码——它将渗透进社会的每一个角落。所有我们称之为"知识工作"的事情，都可以被Tokenmaxxing。这并不意味着我们要淘汰人，而是说人依然需要提供"能动性"。

就像我，我是那个坐在这里，真切关心代数问题的人。我希望那些像当年的我一样、上不起私立学校的孩子能有机会学习。旧金山可能是全美私立学校就读率最高的城市，这不应该是正常现象。你不应该必须有钱才能获得优质教育。这种技术层面的大变革正在发生，而我恰好有一个迫切的需求和强烈的渴望——是那种真实的、揪心的渴望，每当我想到那些十二三岁本该学代数、却被某个官僚或某个道德表演者剥夺了这个机会的孩子，我就感到心痛。

主持人： 所以，在解决自己内心深处的痛点、构建Gary's List的过程中，你逐渐发现了Tokenmaxxing的诸多规律，以及这种全新的构建方式。这也引出了你的下一个项目——GStack。

## GStack的意外诞生

Gary Tan： 我其实根本没有计划做GStack。我只是发现自己一遍又一遍地做重复的事情，厌倦了反复输入同样的内容。于是我打开Apple Notes，把那些我发现自己在Claude Code里反复输入的内容全部记录下来，其实都是很简单的东西。

比如，计划审查（Plan Review）——我开始非常喜欢让Claude生成ASCII艺术图表。我发现有时候Claude会弄混，写出bug，或者任务不完整。但当我说"在开始工作之前，先画一张包含所有数据流、输入输出、用户流程和错误信息的ASCII图"之后，一切都不同了。你可以看到数据流、状态机、依赖关系图、处理管道、决策树。一旦它先生成这张图，就等于把所有上下文都加载进来了，之后它完成任务的完整度就高了很多，也更好地"煮沸了海洋"，并细分成不同的板块，比如架构审查、代码质量、测试等。

在构建Gary's List的过程中，我意识到，自己写代码的时候总是测试做得最少，因为测试实在不好玩。我知道必须有测试，但我更想写有趣的新功能，不喜欢写测试。然后，我也踩了所有人开始"vibe coding"时都会踩的坑——感觉代码很粗糙，对于80%的场景没问题，但只要真实用户一碰，就开始崩。那时我意识到，我可以达到100%的测试覆盖率。不过后来我了解到，100%可能太过了，80%到90%才是目前的最佳实践。





这其实就是Plan Review的第一个版本的雏形。大家都知道"Office Hours"这个技能，就是在尝试打造新产品或新功能时用到的：你怎么知道人们想要这个？它是为谁而做的？它做什么？影响是什么？这就是那个原型技能，当时我甚至不知道"技能"（Skills）这个概念的存在。我把它发出去之后，病毒式传播，大约20万人看到了。

后来我又做了一个功能更丰富的版本，起初叫"Mega Plan"，后来改名为"CEO Plan"。我在这里用到了元提示（meta-prompting）技术。我把另一份评审计划作为基础，然后说："好，做一个这样的版本，但想象一下Brian Chesky坐在你面前。"Brian Chesky有句很有名的话——什么是十星体验？大家通常用二星、三星、四星来衡量酒店，而他会往上走：五星是什么样的？六星呢？七星呢？八星呢？一路往上推演，这是我最喜欢的产品与设计思维练习之一。现在，你每次都可以做这个练习了。

这就是CEO Plan的核心。这个提示词的本质是：找到柏拉图式的理想形态。我特别喜欢其中两点：一是"10倍检验"——什么样的方案更有野心、只需要两倍的努力就能交付十倍的价值？二是从潜在空间（latent space）中发散出的东西能帮助模型真正形象化地理解问题。

我热爱"CEO计划技能"，因为我是一个有ADHD的CEO，我爱潜力，爱那种纯粹的可能性。就这么简简单单的两句话，却能解锁难以置信的能量。GStack就是这样开始的——我只是需要做一些技能卡，然后听说有人在做技能仓库，于是顺势为之。

## 400倍产出背后的工作流

主持人：然后你把这两个技能用得非常频繁，以至于你的Claude实例开始严重积压。聊聊你的具体工作流吧——这是你每天的工作方式，也是你每月能提交数十万行代码的秘诀。

Gary Tan：对，就是这样。过去48小时我提交了13个PR，就像排队一样——任何时候我有新想法，我就进来，用CEO技能，再用测试技能把它做得非常完善，全部在计划模式（Plan Mode）下完成，然后点击批准，Claude就去做所有的事情了。

我这样做了很多次，最后手头堆积了15个不同的功能，全都在排队等待手动测试。这些功能通过了端到端测试、集成测试和单元测试，但最终还是要打开Rails服务器，加载对应用户，配置特定的环境，手动确认一切正常。这个过程让我很烦，于是我尝试用Claude Code的MCP，但每次响应要两秒，用来做QA完全不实用。

我听说微软发布了Playwright，是一种备选测试框架。事后来，当时其实有很多现成的智能体工具可以用。但Claude Code的优点也是它的缺点——它太容易让你"直接开干"了。我就直接进去，输入了大意是"我受够了在Chrome MCP里用Claude Code，太慢了，帮我把微软的Playwright封装起来"，然后按了回车。

GStack就这样慢慢成形了。现在这是我创建新功能的方式。GStack里有CEO、Designer、Developer Experience等角色，还有若干设计工具，最后是Plange。我通常还会运行/codex——最近还加入了/claude in codex。

关于Codex，我从YC校友那里学到的。有一次参加批次活动，大家聊到Claude Code和Codex的比较。当时我是纯Claude Code用户，后来才意识到很多人其实更偏爱Codex——原因是Claude Code非常适合ADHD式的CEO，但有时候Claude模型会出现"一本正经地胡说八道"的情况。聪明是聪明，但毕竟不是最聪明的。所以如果遇到特别复杂的问题，你需要的是那个"200 IQ、近乎沉默"的CTO。

/codex就是GStack的一个技能，它会把你的计划或已有的代码仓库，用命令行提示词交给Codex运行，让它找出所有问题和bug，然后把报告反馈给Claude Code，你再和Claude Code一起解决这些反馈。如果你主要使用Codex作为编码智能体，也可以输入/claude，让Claude短暂客串一下CEO角色。



GStack的工作流程是：先做Office Hours CEO审查，再做设计审查（如果有UI），然后做开发者体验审查（实际上几乎GStack和GBrain的所有内容都需要这步），接着是代码审查，最后用Codex收尾。一旦计划完成、问题都解决了，GStack就会大量调用"向用户提问（Ask User Question）"这个功能——这对我来说至关重要。

这就是人类，无论是"vibe coder"、操作员还是智能体工程师，需要输入自己的判断的地方：我们在构建什么，正在发生什么。这一点没有替代品。如果真的有人能造出一个无需人类在回路中就能自主写软件的东西，我会非常惊讶。我从来不想完全脱离这个回路，我只是希望机器去做我不想做的事情——比如QA。

回到Demo，当我向现代版GStack输入指令时，它会说"哥们，你在干嘛？这个我们已经建过了。"我们有Browse——那是一个有70个命令的长驻进程CLI。QA其实就是Browse，但在QA的提示词里，它会说：检查你的上下文，看看我们在这个分支上做了什么，如果有UI或数据变更，就用浏览器去测试它。这就像拥有了一个黑盒浏览器。第一次看到它真的跑起来的时候，我惊呆了——感觉"微型AGI"已经来了。我知道这不是真正的AGI，真正的AGI意味着不需要我在场。而且说实话，作为一个构建者，我自私地希望机器永远不要完全"搞定"这一切——那样的话，有品位、有技术能力、懂产品反馈、真正了解用户的人，就永远拥有"翅膀"。

## 精简框架，丰满技能（Thin Harness, Fat Skills）

主持人： 我觉得你把很多这些思考都凝练在了X上那篇关于"精简框架，丰满技能"的帖子里。

Gary Tan： 是的，那篇文章其实有一部分是被网上的人嘲讽逼出来的——嘲讽我只是在兜售Markdown。我的实际体会是：Markdown本质上就是代码，只是以不同的方式编译，但你可以用它让计算机做出令人惊叹的事情。

这篇文章的名字其实来自我们的合伙人Pete Koomen。我们内部一直在构建一个智能体——我们称之为"harness（框架）"——然后在用Claude Code的过程中意识到：为什么要反复重写那套框架？我们应该把时间花在"Markdown里应该写什么"这个问题上。

理解Markdown的方式是：想象你是一个婚礼策划师，尝试写下一份清单，告诉下一个接手的人怎么策划婚礼——用平实的语言。所有这些内容都应该写进Markdown。而那些需要确定性执行的操作——比如婚礼策划师可能要打给20个场地——你不会用Markdown来做这件事，你会调用Twilio的API。

智能体工程当今最大的难题，就是人们把本该属于Markdown的逻辑写进了代码里，然后失败了，因为代码是脆弱的，不理解特殊情况，代码本质上不理解你想要什么或者你是谁——它只是在图灵完备的循环里执行确定性的零和一。而现在我们有了LLM，它拥有潜在空间，它知道你是谁，理解你的动机，能处理泛化的情况。

作为工程师，现在很大一部分的魔力就在于搞清楚：哪些部分属于LLM领地，哪些部分属于代码领地？再结合我学到的另一个原则——80%到90%的测试覆盖率——就是说，如果没有测试就把用户扔进去，那就是烂代码，而且比人类写的代码差10倍，因为你完全不知道会发生什么。

所以不只是一要搞清楚潜在空间和确定性空间的边界，还要确保单元测试和集成测试都到位。而"煮沸海洋"的好处在于：机器不在乎工作量，只要你多加投入，你就能达到90%的测试覆盖率，拥有一个虽不完美但95%可靠的系统。

## AI智能体就像法拉利

Gary Tan： 使用OpenClaw就像驾驶法拉利，令人肾上腺素飙升，感觉不可思议。它能搞定你永远不会想到机器能搞定的事，而且速度极快。但它也真的像法拉利——你最好懂点机械。它会在你最需要它的时候抛锚在路边，你得拿着扳手出来，打开引擎盖，自己动手修。



这是计算机科学技术领域非常激动人心的时刻，就像当年的Homebrew Computer Club，就像Apple I问世的那一刻。Jobs和Woz造的Apple I就是一块面包板，字面意义上装在一个用钉子和胶带拼起来的木头盒子里。如果你想要一台个人电脑，那就是你能拿到的东西。而我们现在就处于这样的时刻——有一定技术基础的人，花两三个小时，花500到1000美元在token和云计算上，就能让类似的东西运转起来。一旦跑通，我们就像进入了“套件车法拉利”的阶段——然后你就能驾驶它，去任何你想去的地方。

主持人： 就算是那部分“要自己修车”，很多人也是那种没亲身体验过就不太能理解的感觉。从宏观来看，一切移动得太快了。想想当年，Stack Overflow作为一个能在遇到编程问题时查阅的网站，就已经感觉很神奇了。然后ChatGPT横空出世，比Stack Overflow强多了。但你基本上还是在做同样的事：问问题、复制粘贴代码、运行、再复制粘贴回去。用了Claude Code之后，你终于突破了那个边界，意识到不再需要复制粘贴了——它直接执行并运行代码。哪怕是OpenClaw，我在部署的时候发现确实烦人，它会让自己陷入卡死状态，做一堆讨厌的事。但如果你有Claude Code运行着，它就会去把那些问题修掉。这显然不是长期的最优解，但这种心态转变很重要——不管它多脆弱、需要多少修复，没关系，因为你可以让另一个智能体一直坐在那儿修它。

Gary Tan： 我之前是完全的Claude Code信徒，现在也还是，但构建产品或做智能体工程的时间里，大概只有50%到60%是在Claude Code里了，另外将近一半是通过OpenClaw。

## 个人AI的未来

Gary Tan： 话说回来，我现在大部分时间都在构建GBrain本身。GBrain的起因是我们遇到了Peter，他上了节目。我终于抽出时间去研究OpenClaw了。大约在同一时间，Karpathy写了那篇关于知识型LLM Wiki的文章，于是我想：好，我的代码库里有一堆Markdown，我应该把我所有的上下文都放进那些Markdown里。然后有一天我突然意识到，OpenClaw用的只是GP（向量检索），而GP并不算好——它浪费上下文，把比实际需要多得多的内容加载进上下文窗口。

于是我掉进了一个兔子洞。我进入Conductor，点击快速启动，GStack已经内置在Conductor里了。我就是这样开始的。

其实这个过程比这更有趣。在你写出越来越大的代码库之后，这些知识就加载进了你的大脑。比如，为了给Gary's List构建智能体新闻编辑室，我要真正学习向量嵌入、混合RRF检索和文本分块。当你在里面试图让它工作的时候，你是非常“结果导向”的：我想要的输出是什么样的，文章要达到什么质量，需要哪些引用——你开始建立测试和集成测试，最终拥有一个经过实战检验的产品。

然后我就把两件事联系起来了。这是任何人都可以做到的事。这就是为什么我认为我们正在进入开源的黄金时代。我可以直接在Conductor里打开这个项目，第一步就是让它去看Gary's List的代码，看看我们是怎么做分块、嵌入、混合RRF、RAG这些的，然后把这些提取出来，用Postgres加PG Vector构建一套完整的RAG系统给我的OpenClaw用——就这样，一件事接着一件事，我打开了10个GBrain的窗口，一头扎进去了。

OpenClaw有个很酷的地方，我可以直接问它：我是从什么时候开始用的？——1月23日。还能看到我所有的邮件。我有一条推文说：“Claude Code这周唤醒了我25岁的自己，那个把红牛当水喝、通宵写代码到天亮的我。我们回来了。”建造者的身份重新浮现。我基本上又回到了每天睡四个小时、写代码二十个小时的状态。

主持人： 这也是你开始因为“代码行数”这个话题在网上引发争议的时候，对吧？

Gary Tan： 是的，不过我还是坚持我的观点。

主持人： 用代码行数来衡量开发者生产力这件事在网上确实很有争议——显然有反驳意见说，代码行数根本不能衡量开发者生产力……



Gary Tan：确实不能，但在某种程度上也能。很有意思的是，有人发布了公开的Git工具，可以过滤并标准化"实际逻辑代码行数"。我真的去跑了一下。我因为说"我的编码速度是2013年的100倍"而惹上了麻烦，但做了逻辑行数的精简之后，这个数字反而上去了——实际上是400倍。

显然，这些代码不是我写的，是我指挥15个智能体完成的。在数字层面，这确实削减了一些Claude Code生成的代码行数，但出乎意料的是，它同时削减了我2013年写的代码行数——砍掉了约70%。所以这里有个错位：人类写代码很容易"注水"行数，而Claude Code不会这样做，除非你特别指示它去这么做。它可能构建了错误的东西，或者方向没掌舵好，但它不会为了优化行数而优化，不像在某个岗位上工作的人类那样。

如果你看看2000年乃至1990年代关于软件工程的文献，一个职业软件工程师每天产出的经过测试、生产就绪的代码行数，不是一百行，是五十行，甚至三十行。对我来说可能是十四行——因为我那时是兼职状态。这就是400倍这个数字真正的来源。我本来应该把这个说清楚，而不是继续跟人呛声。如果我在网上拿这个话题刺激过你，我真的道歉——这背后有更深层的逻辑，我最终写了一篇博文详细解释了这一切。

主持人：这不是小事，对技术背景的人来说意义重大，因为它真正提升了你的能力上限。所有攻击你"代码行数"论调的那些人，恰恰是最有可能从彻底放开手脚、全力Tokenmaxxing中获益最多的人。

Gary Tan：对，这是经典的困境：如果你有品位、懂技术，你就是最应该得到这双"翅膀"的人。只需要相信，放下抵触，打开Claude Code，试试看。

主持人：我觉得另一个因素是，体验差异因模型和框架的不同而天差地别。我自己也注意到，任何稍微复杂一点的编程任务，通过OpenClaw智能体来做就会失败——明明用的是完全相同的模型和Opus，但随便超出一个简单脚本的复杂度，效果就不理想了，我就得回到Claude Code。这让我有一种感觉：哦，原来这就是六个月前的感受，就是"这些工具还没到位"的那种感觉。但当我用Opus加Claude Code，感觉就是：它真的到了，而且就要更进一步了。

Gary Tan：我保证，明年这个时候，每个人都会说一件事，而这件事你在这里就先听到了——地球上每一个人都将拥有属于自己的个人AI。我们可以生活在一个拥有自己AI的世界里，有自己的数据、自己的集成、清楚地看到正在发生什么、编写自己的提示词，并且能控制自己看到什么；或者生活在一个企业控制的世界里，就像你的Facebook信息流一样，你不知道算法是谁写的、对谁有利、背后是什么商业模式。

个人计算机革命是人类被馈赠的最强大的礼物，我们正要经历完全相同的转变——个人AI革命。这将是一个选择：人们是否愿意编写自己的提示词？我希望Pete Koomen在这里，因为我们从他那里也学到了这一点：除非你拥有自己的提示词、能为自己而写，否则你就永远活在某个不了解你的PM或开发者定义的API边界以下。他们不会理解你的需求，不会理解你独特的关切。

**这就是那个决定性的问题：你能掌控自己的工具，还是你的工具会掌控你？**

主持人：我觉得公众在这件事上存在一个断层，就是要真正调用这些能力，你需要用到最新最强的模型，而大量烧token的代价目前其实相当高昂。价格在下降，但很多人可能只是在用免费版模型，或者只有最基础的Claude Pro订阅。

Gary Tan：对，也许我们需要正视一点：想要真正进入这种几乎像"接近AGI"的构建境界，你必须大量消耗token。整个Tokenmaxxing的理念，其实让我联想到旧金山的租金。我们经常要对YC创始人说的话就是："旧金山的生活成本高得离谱，但不住在旧金山的机会成本更高。"

在YC早期批次里，我总是遇到有人说："这个公寓每月几千美元，感觉太荒唐了，要不要付？"答案是：必须付。而且不只是在旧金山，还要在Dog Patch这样的社区里，创造那种"缘分式相遇"的可能性。Tokenmaxxing将会是我们需要教导创始人的事情之一，因为它的价值不是一



眼就能看出来的。这就像租金——你应该尽可能地投入，从中获取最大的效用，而不是把它当成办公桌那样可以省则省。在模型使用和token消耗上，你应该大力推进。

主持人：YC的一个核心信条是“活在未来，然后构建缺失的东西”。这是对那句话的深刻诠释——你只需要让自己的大脑真正相信：某天在token上花500美元，只要我在构建真正有价值的东西，就值得。

## 用token换回时间

主持人：Gary，我有个奇特的问题。你觉得，某种程度上，正因为你同时还是YC的CEO，才逼出了这一切吗？你的时间极度稀缺，你不得不想方设法用零散的会议间隙完成数十万行代码，而如果是一个全职工程师，可能会花时间慢慢打开网站，点来点去测试一下。你的时间如此宝贵，所以你不断逼自己把一切都自动化。

Gary Tan：有时候我真的很羡慕那些“时间亿万富翁”。我看着我的孩子们——这些孩子现在就是时间亿万富翁，你可以做任何你想做的事，学习任何你想学的东西，太棒了。我们在Startup School上不断遇到这样的年轻人，他们就是时间亿万富翁。

我个人的哲学是：我的脑子里一直在疯狂飞奔，感觉要活完这个身体里的100亿辈子，每一刻都必须有意义。如果你能Tokenmaxxing，那就像购买了数百万年的机器意识。现在我也能成为时间亿万富翁了——不是用我自己的时间，而是用机器的时间，为我在乎的人和我在乎的事业工作：YC，以及让构建者能够构建。

在我们去年很多内部会议和团队务虚会上，我们都在讨论：怎么教下一代使用这些工具？我希望我能说这一切都是精心设计的宏大计划，但事实不是。不过从潜意识层面看，我相信Lightcone一集集的录制，以及坐在Boris Chernny旁边的那个时刻，对我影响深远——他说出了一些让我意识到“我自己就能做到”的话。他说他的团队不写一行代码，我就想：哦，我其实也能这样，在场的观众们也一样——你们和我没有任何本质区别。我们都是从同一个起点出发的。

我不觉得自己已经高不可攀，只是一个努力做事的人。当我坐在Boris旁边，我知道他是我见过的最优秀的工程师之一，但如果我们都打开同一个提示词，我们拥有的是同一个提示词，同一台MacBook Pro。没有任何东西阻挡我们任何一个人，去调用可能是数百万年的token来服务人类。

主持人：Gary，我觉得这最后这句话值得直接发到X上。

Gary Tan：你可以向机器借时间，从而拥有无限的时间。

主持人：这个时代真好，多么美好的结语。感谢Gary带我们看见了未来。

Gary Tan：谢谢你们。

主持人：好的，感谢收看，我们下期Lightcone见。

## 风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



限时权益申领

\* 体验资格每人限申领一次

扫码

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

298 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论



reclard  
5小时前 中国安徽省

GBrain是个非常cool的东西。个人好用，企业也可根据实际所需，工程化后应用。

0 回复